Web Application Security Assessment Report

Company	**Remonixa Technology**
Prepared By	**Security Team**
Assessment Date	**20-04-2026**

Confidential – Sample Report

1. Executive Summary

This security assessment was conducted on the DVWA (Damn Vulnerable Web Application) platform to identify and document common web application vulnerabilities. The assessment employed manual testing methodologies combined with automated vulnerability scanning tools, following industry-standard OWASP (Open Web Application Security Project) testing practices.

The DVWA platform, as expected from a deliberately vulnerable application, contains multiple critical security flaws that could allow attackers to gain unauthorized access, steal sensitive data, and manipulate application logic.

Testing Methodology: All tests were performed in a controlled, isolated environment using manual penetration testing techniques and industry-standard security tools. No attempts were made to access external systems or live environments.

Total Vulnerabilities Identified: 4 Major Issues

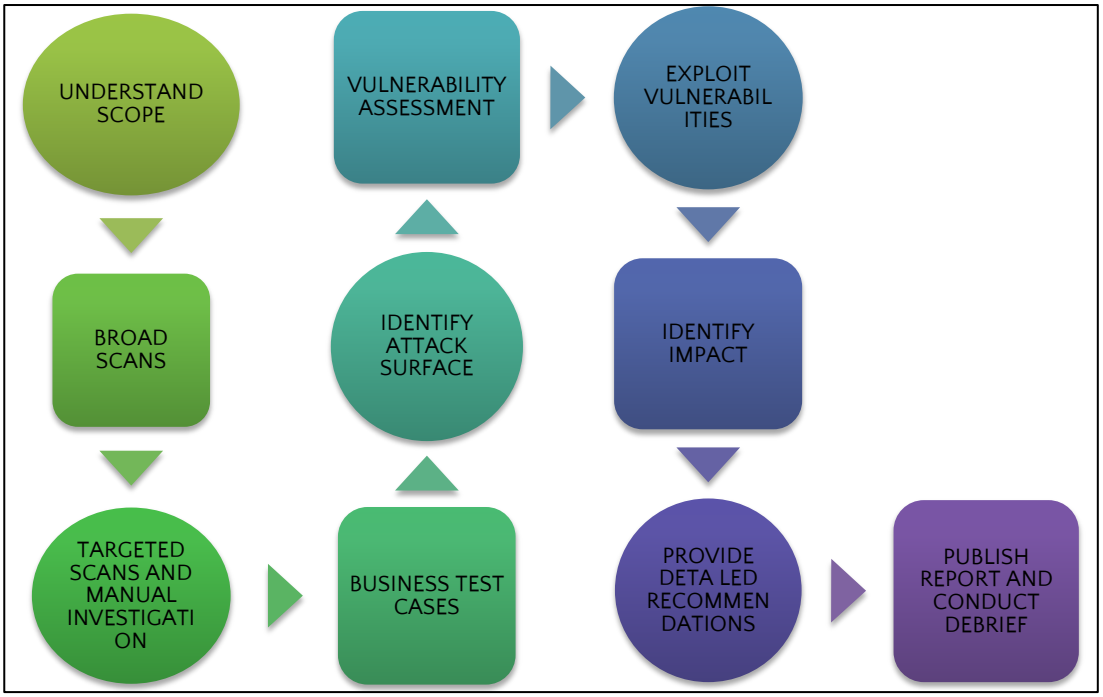
2. Scope & Methodology

- **Scope of Application Tested**

The scope of this penetration test was limited to the URL mentioned below:

Target Application	DVWA (Damn Vulnerable Web Application) v1.10
Application URL	https://hidden-target-url.com
Testing Environment	Local Test Environment
Database	MySQL

- **Testing Methodology / Approach**



- **STANDARDS AND FRAMEWORK FOLLOWED**

Our testing methodology adapts from the following security frameworks and vulnerability categories:

1. Open Web Application Security Project Framework (OWASP)
2. Web Application Security Consortium (WASC)
3. The Open-Source Security Testing Methodology Manual (OSSTMM)

- **SEVERITY RATING**

Organization Name follows the Common Vulnerability Scoring System (CVSS) scoring system to rate vulnerabilities.

The CVSS assessment measures three areas of concern:

- Base Metrics for qualities intrinsic to a vulnerability
- Temporal Metrics for characteristics that evolve over the lifetime of vulnerability
- Environmental Metrics for vulnerabilities that depend on an implementation or environment

For more information on what the Base, Temporal, and Environment Metrics in the CVSS scoring system are, please visit https://www.first.org/cvss/specification-document\

CVSS Severity Rating	CVSS Score
Critical	9.0 - 10.0
High	7.0 - 8.9
Medium	4.0 - 6.9
Low	0.1 - 3.9
None	0.0

Severity levels are determined based on:

- Exploitability
- Impact on confidentiality
- Impact on integrity
- Impact on availability

4. Vulnerability Summary Table

The Web Vulnerability Assessment and Penetration Testing (VAPT) conducted for the target organization yielded important findings and insights. This summary provides an overview of the key results obtained during the assessment. It was observed that the application was exposed to a total of 4 security vulnerabilities during the given assessment.

Sr. No	Vulnerability Identified	Severity	Revalidation Status	CVSS Score	OWASP Top 10 Category
1	SQL Injection	Critical	Open	9.1	A05:2025 – Injection
2	Stored Cross-Site Scripting (XSS)	Medium	Open	6.1	A05:2025 – Injection
3	Authentication Issues	High	Open	8.8	A07:2025 - Authentication Failures
4	Authorization Issues	High	Open	8.8	A01:2025 - Broken Access Control

5. Technical Vulnerability Report:

5.1 Vulnerability Name: SQL Injection – Data Extraction

During testing, the application was found to be vulnerable to SQL Injection. SQL Injection is a code injection technique that allows attackers to insert malicious SQL statements into entry fields. The application fails to properly validate and sanitize user inputs before using them in database queries.

Impact:

- Bypass authentication mechanisms
- Extract sensitive data
- Modify/delete database records
- Escalate privileges

Affected Endpoint: Login Page and User ID Search Functionality.

Steps to Reproduce:

Step 1: Intercept the request using Burp Suite and navigate to the vulnerable endpoint. Submit a normal input in the id parameter.

GET /DVWA/vulnerabilities/sqli/?id=1&Submit=Submit HTTP/1.1

Step 2: Inject the payload into the **id parameter**: **1' OR '1'='1**

Step 3: Use the payload to determine the number of columns in the backend query:

1' ORDER BY 1,2-- -

Step 4: After identifying the column structure, a UNION-based SQL Injection payload was used to retrieve sensitive information.

Payload used: **1' UNION SELECT user,password FROM users—**

Proof Of Concepts:

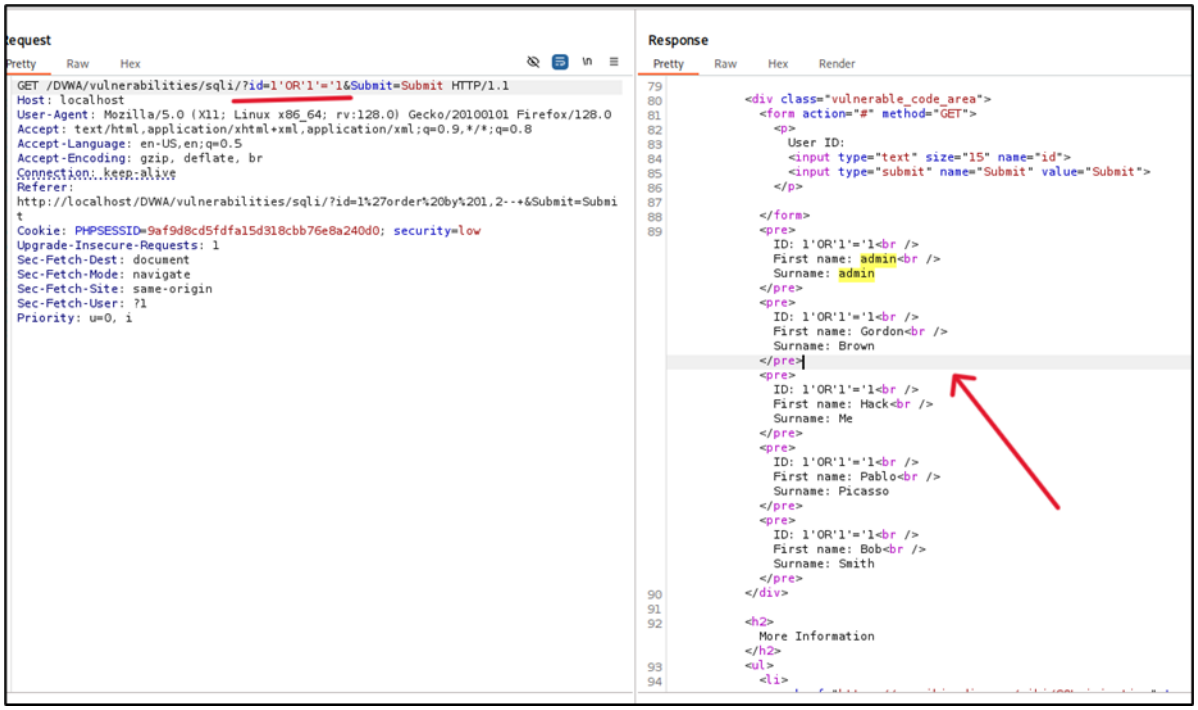


Fig 1. The application returned multiple user records instead of a single result, confirming SQL injection.

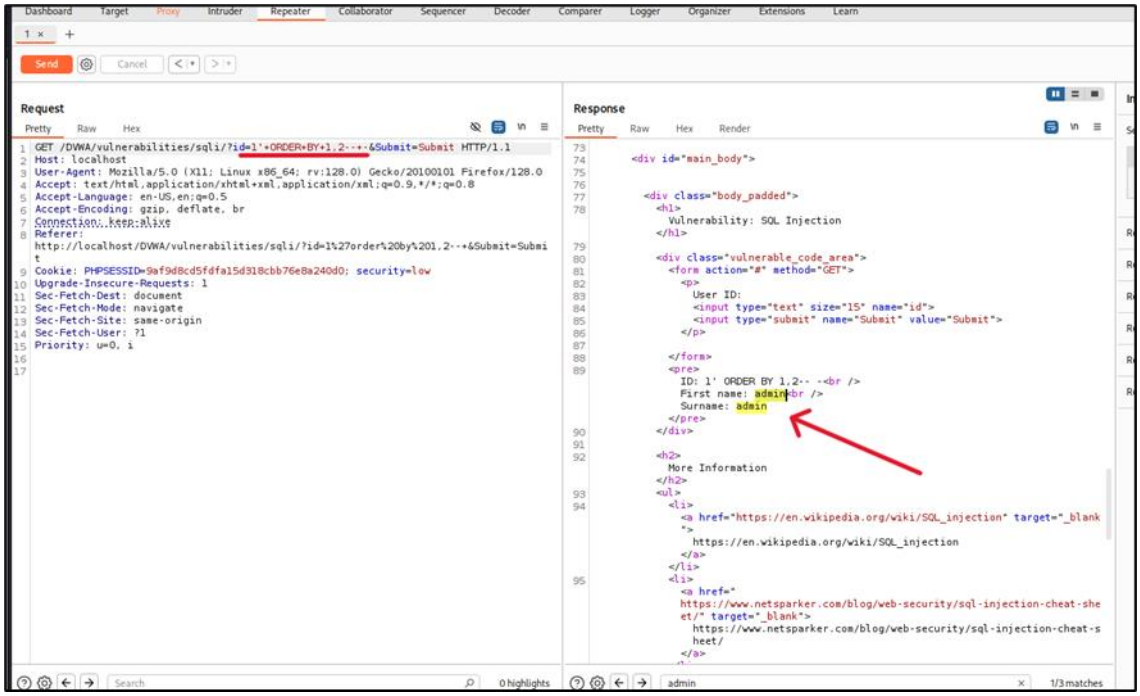


Fig2. Column enumeration

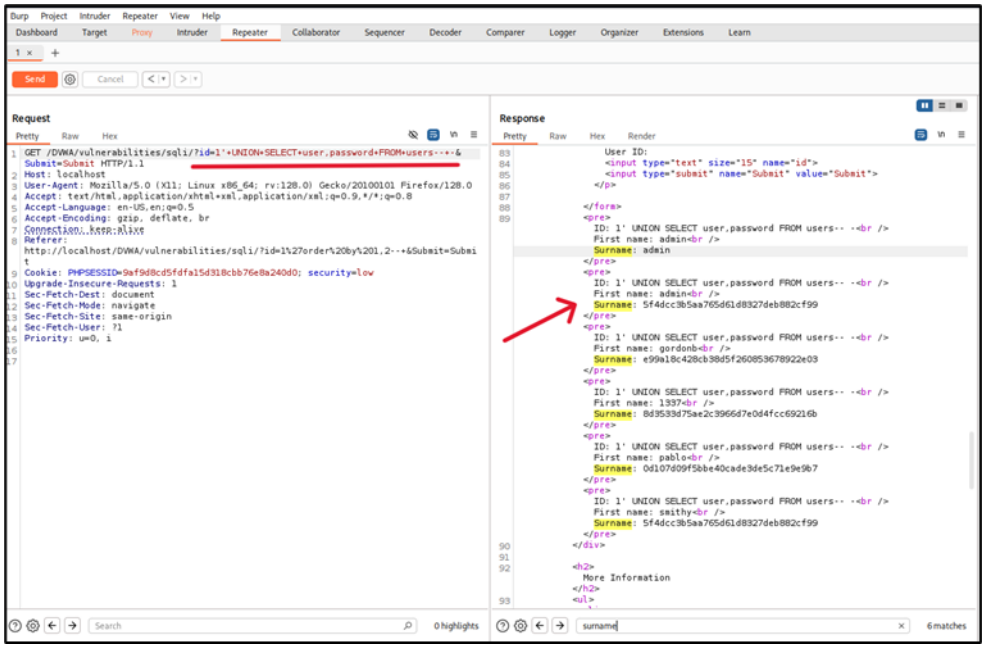


Fig3. Data extraction from database

Observed Result:

The SQL injection vulnerability was successfully exploited by injecting malicious SQL queries into the id parameter. The application responded with database records containing usernames and password hashes, confirming the vulnerability.

Example extracted data (partially masked):

admin : 5f4dcc3b5aa765d61d8327deb****

gordonb : e99a18c428cb38d5f260853678****

Expected Result:

- Only return the exact User ID match
- Prevent SQL syntax manipulation
- Return error for invalid queries

Real-World Attack Scenario:

- Attacker identifies dynamic SQL queries in application.
- Attacker crafts malicious SQL payload
- Application fails to validate input and executes modified query
- Attacker gains unauthorized database access
- Attacker steals all user credentials
- Attacker accesses other systems or sells data

Severity: Critical

The vulnerability is rated Critical because the application accepts unsanitized user input in the id parameter, allowing attackers to execute arbitrary SQL queries. Exploitation demonstrated unauthorized access to sensitive database records, including usernames and password hashes.

Recommendations:

- Use Prepared Statements Parameterized Queries
- Input Validation, whitelist acceptable characters
- Output Encoding (HTML encode before displaying)
- Database Security -use least privilege accounts
- Implement Web Application Firewall (WAF) rules
- Code Review and secure coding standards

5.2 Vulnerability Name: Stored Cross-Site Scripting (Stored XSS)

Stored Cross-Site Scripting (Stored XSS) is a web application vulnerability that allows attackers to inject malicious scripts into a web application's stored data. These scripts are permanently stored on the server (for example, in a database) and are executed whenever a user accesses the affected page.

Impact:

Successful exploitation of this vulnerability could allow an attacker to:

- Execute arbitrary JavaScript in the victim’s browser
- Steal session cookies and hijack user sessions
- Perform actions on behalf of authenticated users
- Redirect users to malicious websites
- Deface the application interface
- Deliver phishing attacks

Because the payload is stored on the server, every user who accesses the affected page may be impacted.

Affected Endpoint: XSS (Stored) page - Message input field

Steps to Reproduce:

1. Navigate to the vulnerable page: http://localhost/DVWA/vulnerabilities/xss_s/
2. Enter the random details in the guestbook form.
3. Enable Burp Suite Intercept and submit the request and Capture the request in Burp Suite Proxy.
4. Modify the txtMessage parameter by injecting the payload and forward the modified request to the server.
5. Refresh the page or view the stored guestbook message and the injected script executes and displays an alert box.

Proof Of Concept:

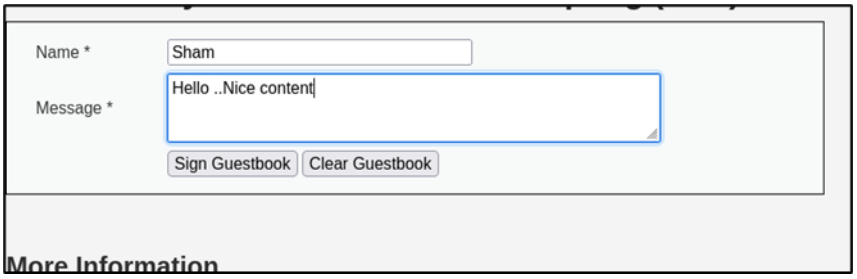


Fig1. Vulnerable Guestbook Page

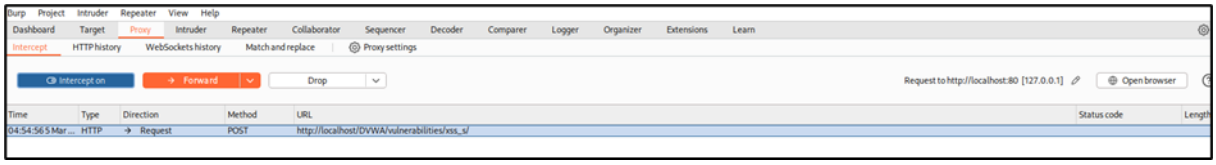


Fig2. Intercepted Request in Burp Suite Proxy

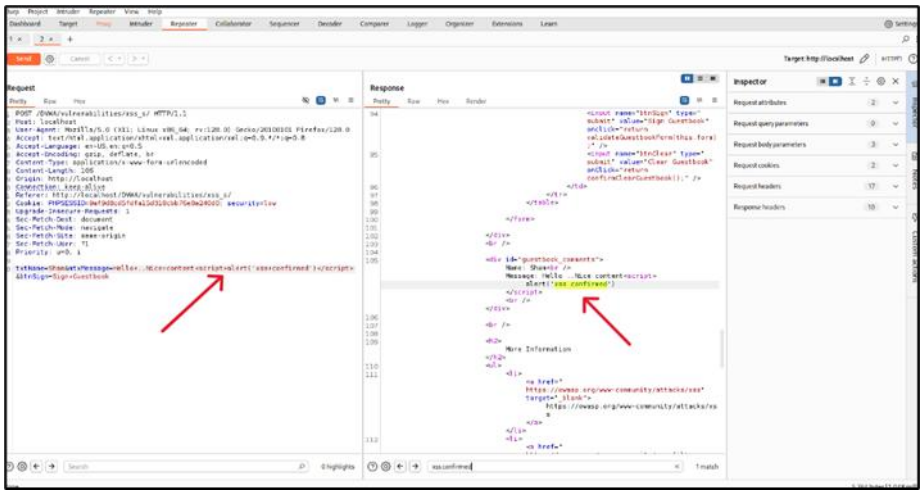


Fig3. Response showing stored script

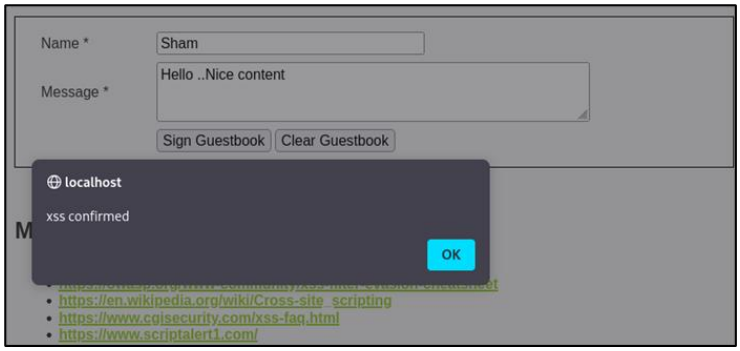


Fig.4 XSS Alert Popup and this confirms the presence of a Stored XSS vulnerability.

Observed Result:

In this case, the application fails to properly validate and sanitize user input submitted through the guestbook message field. An attacker can inject malicious JavaScript code which gets stored in the application and executed when the page is viewed.

During testing, the application accepted and stored a JavaScript payload without filtering. When the stored content was rendered in the browser, the injected script executed successfully, demonstrating the presence of a Stored XSS vulnerability.

This vulnerability could allow attackers to execute arbitrary JavaScript in the victim's browser, potentially leading to session hijacking, credential theft, or other malicious actions.

Expected Result:

The application should validate and sanitize user input properly. Malicious JavaScript code should be blocked, removed, or displayed as plain text instead of being executed. When the page is viewed, no injected script should run, ensuring the application is protected from Stored XSS attacks.

Real-World Attack Scenarios (Stored XSS):

- Session Hijacking – Malicious script stored in the application steals session cookies when users view the infected page.
- Persistent Phishing – Attacker injects fake login forms or warning messages that appear to every visitor.
- Stored Payload Affecting Multiple Users – Since the script is stored on the server, it impacts every user who visits the vulnerable page, making it more dangerous than reflected XSS.

Severity: Medium

This vulnerability is rated Medium because the application stores unsanitized user input, allowing attackers to inject malicious scripts that execute in a user's browser. Successful exploitation may lead

to session hijacking, credential theft, or unauthorized actions. Exploitation requires user interaction, which reduces the overall severity.

Recommendation:

To mitigate Stored XSS vulnerabilities, the following security measures should be implemented:

- Input Validation: Validate all user inputs on both client-side and server-side to prevent malicious data from being processed.
- Output Encoding: Encode user input before displaying it in the browser to prevent script execution.
- Secure Framework Functions: Use secure framework functions or templating engines that automatically escape user input.
- Content Security Policy (CSP): Implement a strong Content Security Policy to restrict execution of unauthorized scripts.
- Input Sanitization: Sanitize user inputs to remove or neutralize malicious HTML and JavaScript code.
- Security Libraries: Use security libraries such as OWASP Java Encoder or DOM Purify to safely handle user-generated content.

5.3 Vulnerability Name: Authentication Failure – Brute Force Vulnerability

Authentication Failure is a vulnerability where the application does not properly protect its authentication mechanism, allowing attackers to bypass or abuse login controls to gain unauthorized access. This can happen due to weaknesses such as weak passwords, missing account lockout, lack of rate limiting, poor session handling, or susceptibility to brute force attacks.

Impact:

- Compromise user accounts
- Gain unauthorized access
- Escalate privileges (if administrative credentials are guessed)
- Access sensitive data
- Use compromised accounts for further attacks

Affected Endpoint: Login Authentication Functionality

Steps to Reproduce:

1. Log in to Damn Vulnerable Web Application.
2. Intercept the login request using Burp Suite Proxy.
3. Send the request to Intruder.
4. Mark the username and password parameters as payload positions.

5. Load user and password wordlists:
    - http_default_users.txt
    - http_default_pass.txt
6. Launch the Intruder attack (Cluster Bomb).
7. Analyse responses and identify valid credentials based on differing response length or successful login response.

Proof Of Concept:

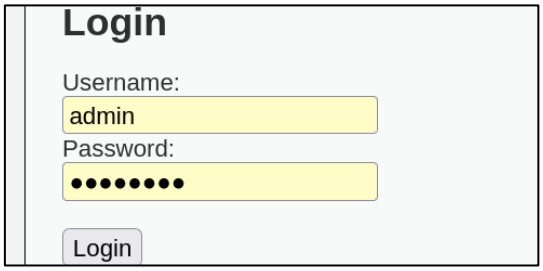


Fig1 . Login Page

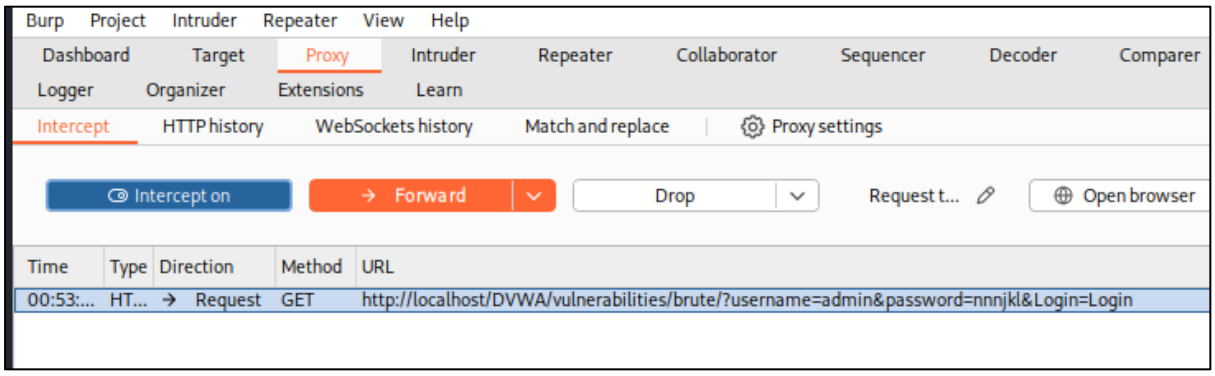


Fig2 .Captured the login request in Burp Suite.

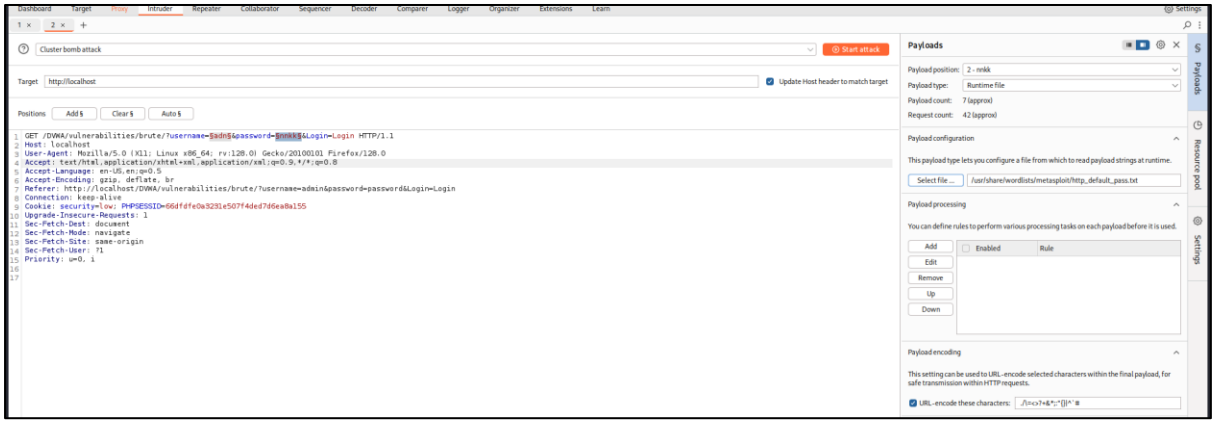


Fig 3. Configured Burp Intruder with username and password payloads.

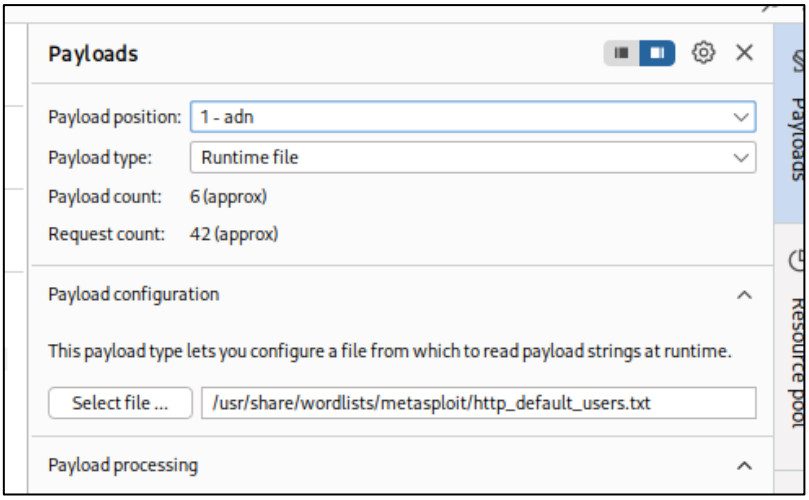
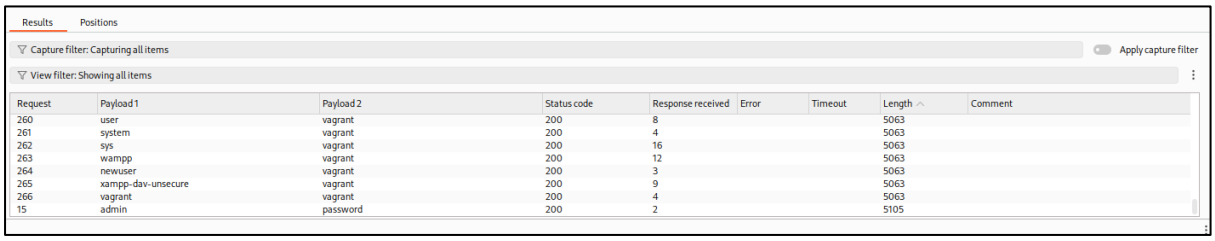


Fig 4. Performed brute force attack using multiple credential combinations.



Results Positions

Capture filter: Capturing all items

View filter: Showing all items

Apply capture filter

Request	Payload 1	Payload 2	Status code	Response received	Error	Timeout	Length	Comment
260	user	vagrant	200	8			5063	
261	system	vagrant	200	4			5063	
262	sys	vagrant	200	16			5063	
263	wampp	vagrant	200	12			5063	
264	newuser	vagrant	200	3			5063	
265	xampp-dav-unsecure	vagrant	200	9			5063	
266	vagrant	vagrant	200	4			5063	
15	admin	password	200	2			5105	

Fig5. Identified valid credentials through a successful login response.

Observed Result:

The application accepted unlimited authentication attempts without:

- Account lockout
- Rate limiting
- Temporary blocking
- CAPTCHA challenge
- Brute force detection controls

Expected Result:

The application should prevent brute force attacks by implementing controls such as:

- Account lockout after repeated failures
- Rate limiting/throttling
- CAPTCHA after multiple attempts
- Multi-factor authentication (MFA)
- Monitoring and alerting for abnormal login activity

Severity: High

This vulnerability is rated High because the application allows brute-force attacks due to weak authentication controls and lack of protection against repeated login attempts.

Recommendation:

- Implement account lockout after failed attempts.
- Enforce rate limiting on login requests.
- Use CAPTCHA after repeated failures.
- Enforce strong password policies.
- Implement Multi-Factor Authentication (MFA).
- Monitor and alert on brute force patterns.
- Restrict automated login attempts using IP-based controls.

5.4 Vulnerability Name: Authorization Bypass (Broken Access Control)

The application is vulnerable to Authorization Bypass, where a non-administrative user can access functionality intended only for privileged users. During testing, logging in with lower-privileged user credentials provided unauthorized access to the user management interface, allowing access to protected administrative functionality.

Affected Endpoint: User Management Functionality (Admin-Restricted Access Control Mechanism)

Impact:

- Access restricted administrative functions
- View or modify other users' data
- Perform unauthorized actions
- Escalate privileges
- Compromise confidentiality and integrity of the application

Steps to Reproduce:

1. Navigate to the Authorisation Bypass module in Damn Vulnerable Web Application.
2. Log in using non-admin user credentials (gordonb / abc123).
3. Access the restricted user management functionality.
4. Observe access to admin-only features without proper authorization checks.

Proof of Concept (PoC):

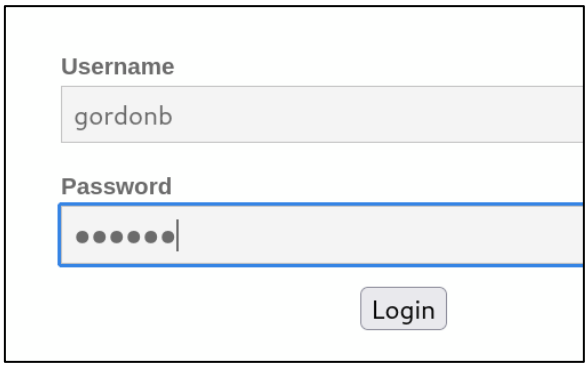


Fig1. Logged in using a non-admin user account to test authorization controls.

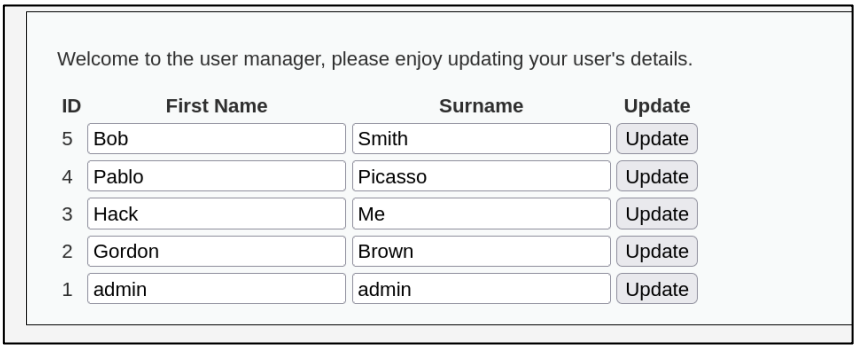


Fig2. Accessed admin-restricted user management functionality using a low-privileged user.

Observed Result:

The application granted a non-privileged user access to restricted administrative functionality and allowed unauthorized interaction with protected resources.

Expected Result:

The application should enforce proper authorization checks and restrict access to administrative functionality only to authorized users.

Severity: High

This vulnerability allows unauthorized users to bypass access controls and access privileged functionality, potentially leading to privilege escalation, unauthorized modifications, and compromise of sensitive data.

Attack Scenario:

An attacker logs into the application using a standard user account and discovers that administrative functionality is accessible without proper role validation. The attacker accesses the user management panel, modifies user records, escalates privileges, or abuses administrative functions to compromise the application.

Recommendation:

- Implement server-side role-based access control (RBAC).
- Enforce authorization checks for every privileged function.
- Deny access to restricted resources for non-admin users.
- Validate user roles before allowing sensitive actions.
- Conduct access control testing for privilege escalation scenarios.

6. Risk Analysis:

Overall Application Risk Level: CRITICAL

The DVWA platform, in its current configuration, presents a CRITICAL risk to information security. The application contains multiple high-severity vulnerabilities that could lead to:

- Complete data breach and unauthorized access
- Unauthorized modification or deletion of critical data
- Authentication bypass and account takeover
- Regulatory compliance violations (GDPR, PCI-DSS, HIPAA)
- Reputational damage and loss of customer trust

7. Recommendation Summary:

1. SQL Injection: Implement parameterized queries (prepared statements), input validation, and proper output handling to prevent injection attacks.
2. Cross-Site Scripting (XSS): Apply input sanitization, output encoding, and implement Content Security Policy (CSP) to mitigate XSS vulnerabilities.
3. Authorization Issues: Enforce strong server-side access controls, role-based access control (RBAC), and validate user permissions for every sensitive action.
4. Authentication Issues: Implement account lockout, rate limiting, strong password policies, and multi-factor authentication (MFA) to strengthen authentication security.

8. Conclusion:

The assessment identified multiple high-severity vulnerabilities, including SQL Injection, Stored Cross-Site Scripting (XSS), Authorization Issues, and Authentication Issues, which

remain open and require remediation. These vulnerabilities could lead to unauthorized access, data compromise, and application misuse if exploited. It is recommended to address the identified issues and perform revalidation after remediation to confirm that appropriate security controls have been effectively implemented.

9. Disclaimer:

This is a sample security report by Remonixa Technology for demonstration purposes using test environments only.

Prahlada Thumabalabeedu
CEO & Founder
Remonixa Technology

.